

Tytuł projektu: Fortuna – aplikacja dla klientów banku

Frontend: React, Vite, TypeScript, Redux Toolkit, TanStack Query

Backend: ASP.NET Core, .NET, Entity Framework Core

Bazy danych: SQL Server, osobna baza zapisu i baza odczytu

Architektura: Clean Architecture, DDD, CQRS, Outbox, Read Model

Autor 1: Jakub Wołczko

Nr albumu: 70210

Autor 2: Jakub Łaszuk

Nr albumu: 72418

Repozytorium: [GitHub – Designing multi-tier business applications](#)

Spis treści

1	Cel i zakres systemu	4
2	Widoki aplikacji	4
2.1	Landing page	4
2.2	Rejestracja użytkownika	5
2.3	Logowanie	6
2.4	Dashboard klienta	6
2.5	Przelewy	7
3	Architektura systemu	7
3.1	Warstwy backendu	8
3.2	CQRS i rozdzielenie zapisu od odczytu	10
4	Model domenowy	10
4.1	Agregaty i value objecty	11
4.2	Zdarzenia domenowe	11
5	Działanie backendu	11
5.1	Rejestracja i logowanie	11
5.2	Pobieranie dashboardu	12
5.3	Wykonywanie przelewu	13
5.4	Symulacja przelewu przychodzącego	14
6	Mechanizm Outbox i projekcje	14
7	Schematy baz danych	15
7.1	Baza zapisu	15
7.2	Baza odczytu	16
7.3	Najważniejsze relacje	16
8	Frontend	17
8.1	Struktura	17

8.2	Komunikacja z API	17
9	Bezpieczeństwo i walidacja	17
10	Podsumowanie	18

1 Cel i zakres systemu

Fortuna jest demonstracyjną aplikacją bankową dla klientów indywidualnych. System pozwala użytkownikowi założyć konto, zalogować się, obejrzeć dashboard produktów finansowych oraz wykonać przelew. Aplikacja została zaprojektowana jako system wielowarstwowy, w którym interfejs użytkownika komunikuje się z backendem przez REST API, a backend rozdziela model zapisu od modelu odczytu.

Zakres funkcjonalny obejmuje:

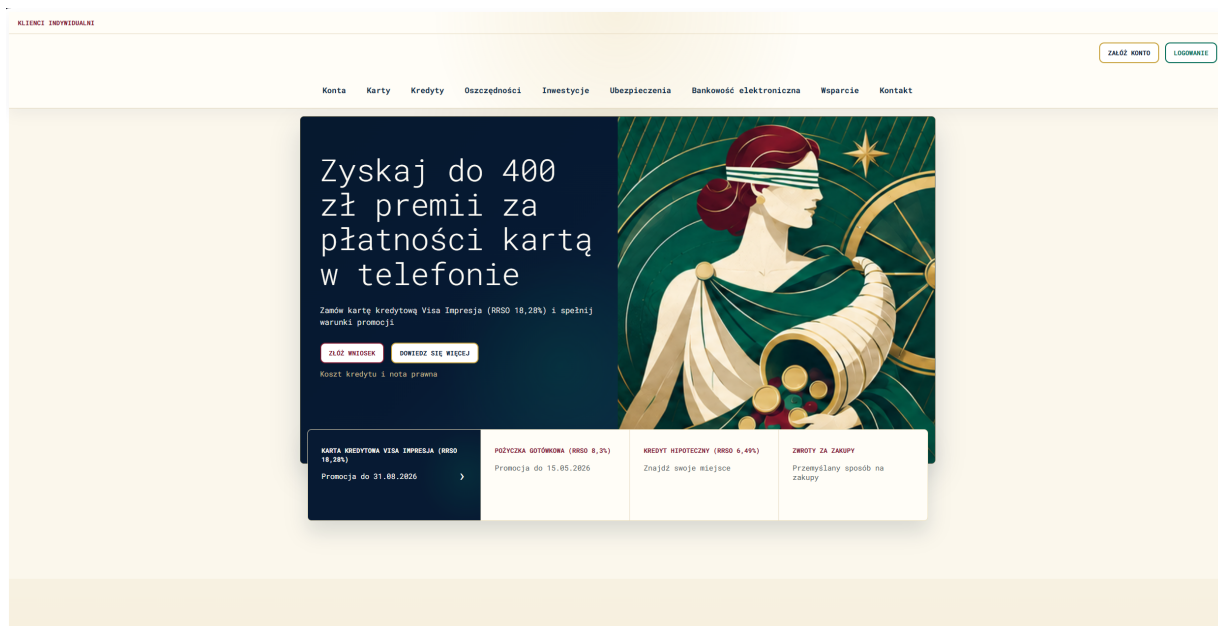
- stronę landing page z ofertą banku,
- rejestrację klienta oraz automatyczne utworzenie produktów startowych,
- logowanie z wykorzystaniem tokenu JWT,
- dashboard z podsumowaniem środków, listą produktów i historią zdarzeń,
- tworzenie przelewów własnych i zewnętrznych,
- symulację przelewu przychodzącego przez endpoint API,
- asynchroniczną aktualizację modelu odczytu przez mechanizm Outbox.

2 Widoki aplikacji

Frontend jest aplikacją SPA napisaną w React. Routing definiuje trzy ścieżki publiczne: landing page, logowanie i rejestrację oraz ścieżkę chronioną dashboardu. Po zalogowaniu token i identyfikator klienta są przechowywane po stronie klienta i używane do autoryzowanych wywołań API.

2.1 Landing page

Landing page prezentuje ofertę promocyjną banku, główną grafikę nawiązującą do Fortuny oraz kafle ofertowe. Z poziomu tego widoku użytkownik może przejść do rejestracji albo otworzyć modal logowania.



Rysunek 1: Landing page aplikacji Fortuna.

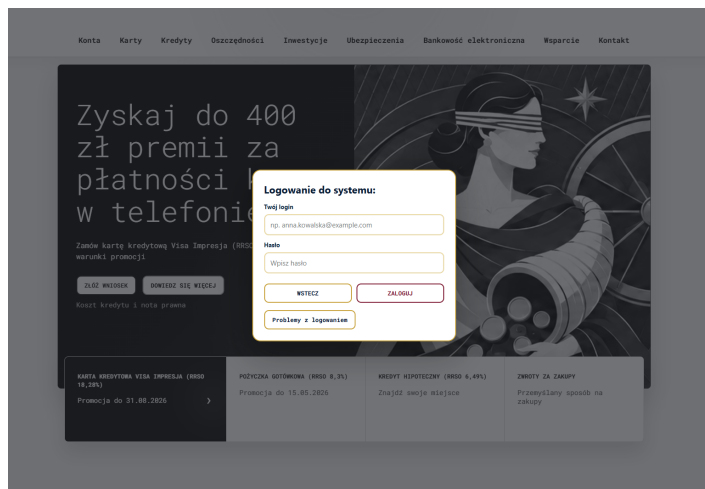
2.2 Rejestracja użytkownika

Formularz rejestracji zbiera imię, nazwisko, adres e-mail, hasło oraz typ klienta. Po przesłaniu formularza frontend wywołuje endpoint `POST /api/customers`. Backend waliduje dane, hashuje hasło i tworzy klienta razem z domyślnymi produktami.

Rysunek 2: Ekran rejestracji klienta.

2.3 Logowanie

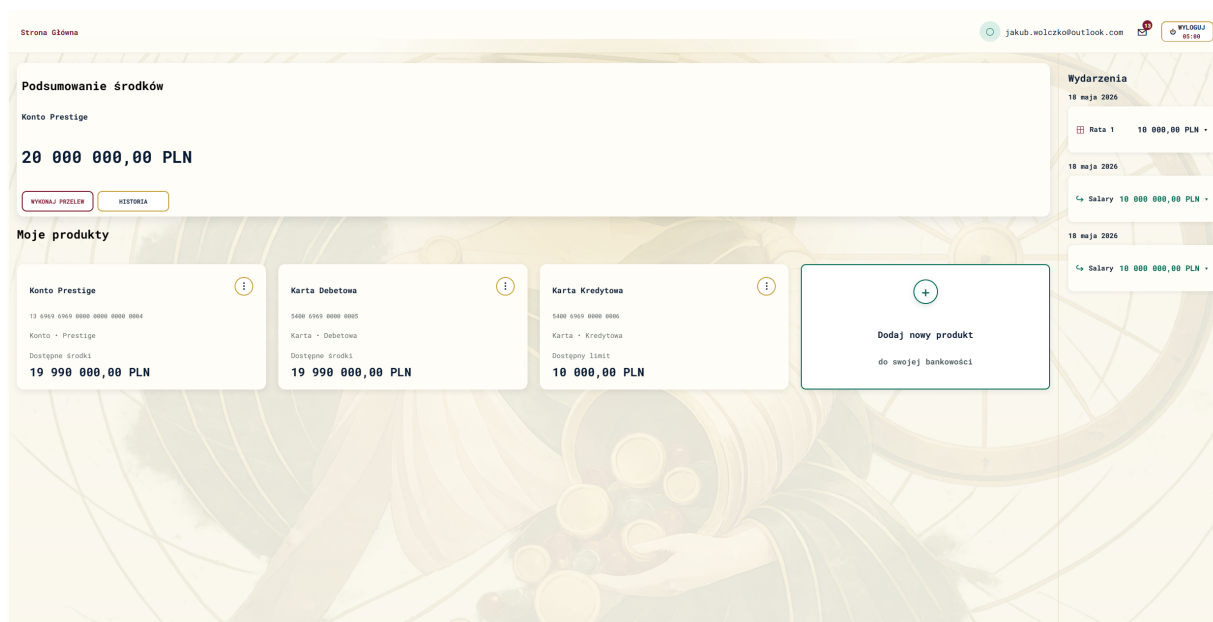
Logowanie odbywa się przez endpoint `POST /api/customers/login`. W odpowiedzi API zwraca token JWT oraz datę wygaśnięcia. Token jest następnie dołączany w nagłówku `Authorization: Bearer ...` do żądań dashboardu i przelewów.



Rysunek 3: Popup logowania.

2.4 Dashboard klienta

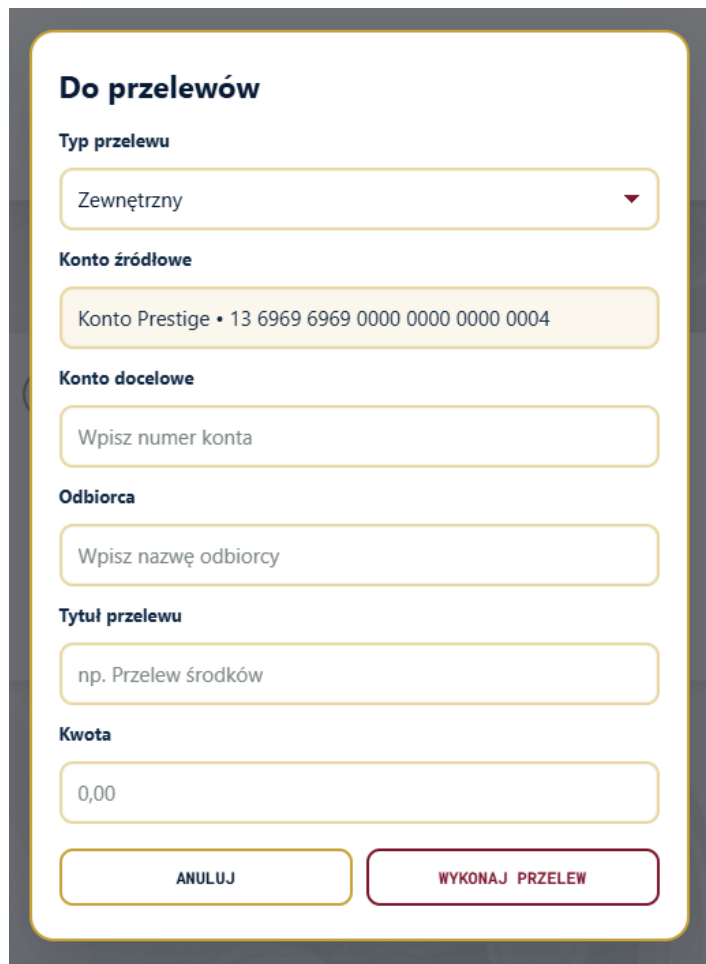
Dashboard pokazuje łączną wartość środków, produkty klienta, skrót zdarzeń oraz akcje bankowe. Dane są pobierane z endpointu `GET /api/dashboard/{customerId}`, który korzysta z osobnego modelu odczytu.



Rysunek 4: Dashboard klienta po zalogowaniu.

2.5 Przelewy

Popup przelewu pozwala wybrać typ przelewu, konto źródłowe, odbiorcę, tytuł i kwotę. Dla przelewów własnych wymagany jest produkt docelowy klienta, a dla przelewów zewnętrznych numer konta i nazwa odbiorcy.



Do przelewów

Typ przelewu

Zewnętrzny ▼

Konto źródłowe

Konto Prestige • 13 6969 6969 0000 0000 0004

Konto docelowe

Wpisz numer konta

Odbiorca

Wpisz nazwę odbiorcy

Tytuł przelewu

np. Przelew środków

Kwota

0,00

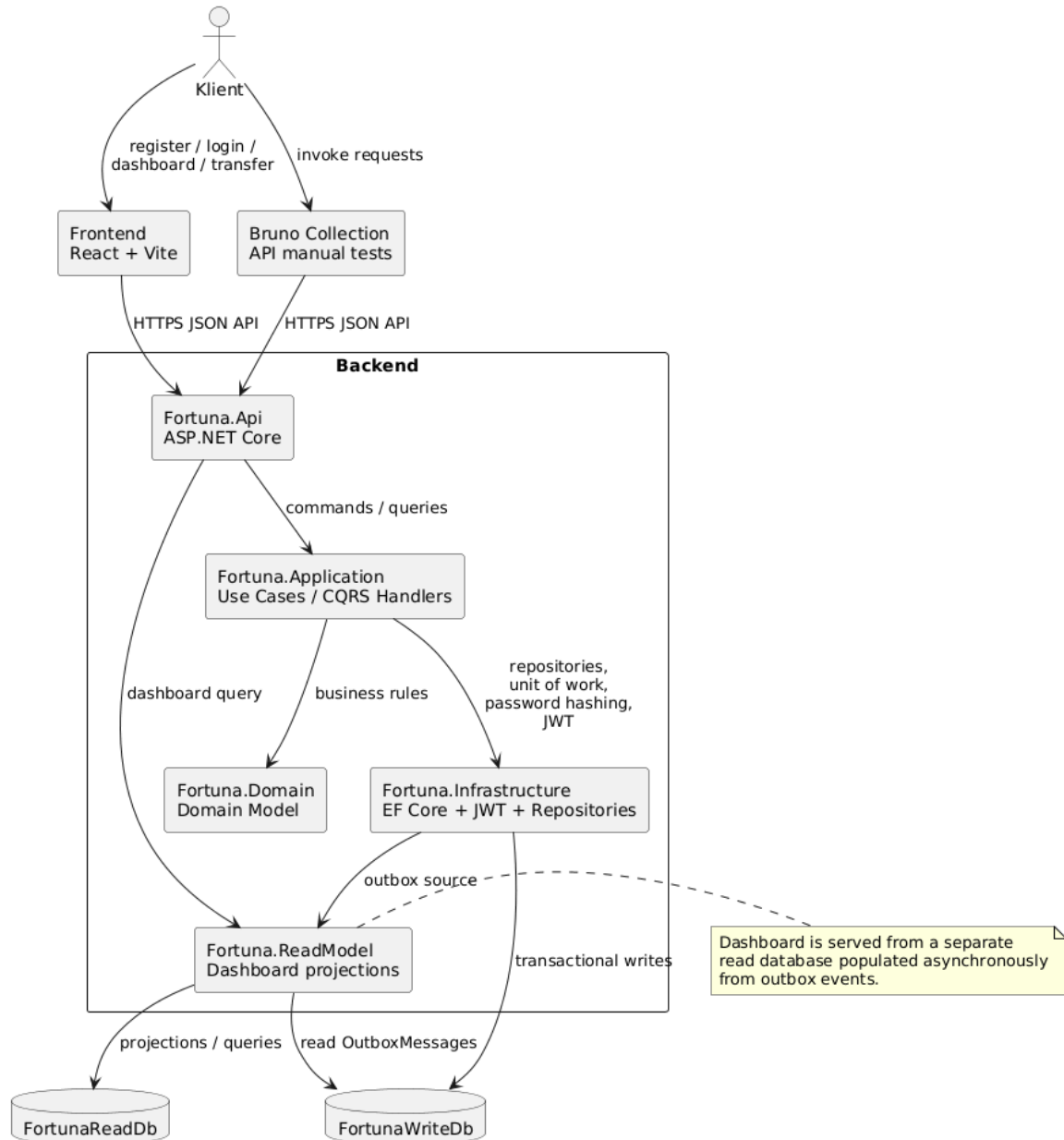
ANULUJ WYKONAJ PRZELEW

Rysunek 5: Formularz wykonywania przelewu.

3 Architektura systemu

System składa się z aplikacji frontendowej, backendu ASP.NET Core oraz dwóch logicznych modeli danych: zapisu i odczytu. Backend jest podzielony na projekty odpowiadające warstwom Clean Architecture.

Fortuna - System Context

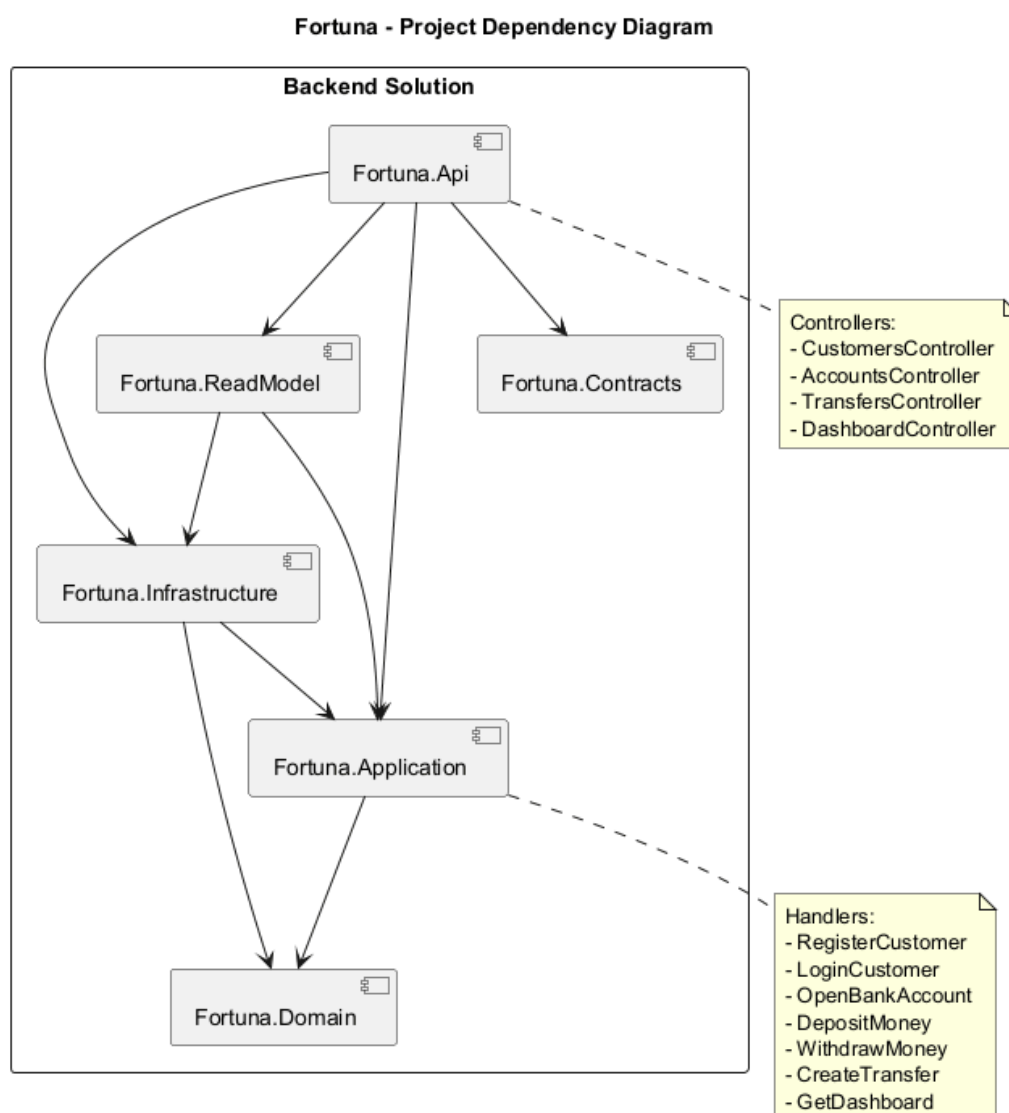


Rysunek 6: Kontekst systemu Fortuna.

3.1 Warstwy backendu

Projekt	Odpowiedzialność
Fortuna.Api	Warstwa HTTP. Zawiera kontrolery, konfigurację CORS, Swagger, JWT, middleware obsługi wyjątków i mapowanie endpointów.
Fortuna.Application	Przypadki użycia systemu. Zawiera komendy, zapytania, handlers CQRS, abstrakcje repozytoriów, walidację scenariuszy i DTO dashboardu.

Projekt	Odpowiedzialność
Fortuna.Domain	Model domenowy DDD: agregaty, encje, value objecty, reguły biznesowe i zdarzenia domenowe.
Fortuna.Infrastructure	Implementacje techniczne: EF Core dla bazy zapisu, repozytoria, Unit of Work, hashowanie haseł, JWT, Outbox processor.
Fortuna.ReadModel	Baza odczytu dashboardu, projekcje zdarzeń domenowych i repozytorium zapytań dashboardu.
Fortuna.Contracts	Kontrakty API używane przez kontrolery: requesty i response'y.



Rysunek 7: Zależności projektów backendu.

Zależności są skierowane do wewnątrz. Warstwa domenowa nie zależy od infrastruktury ani API. Warstwa aplikacyjna zna domenę i abstrakcje, ale nie zna szczegółów EF Core czy JWT. Infrastruktura implementuje interfejsy wymagane przez Application.

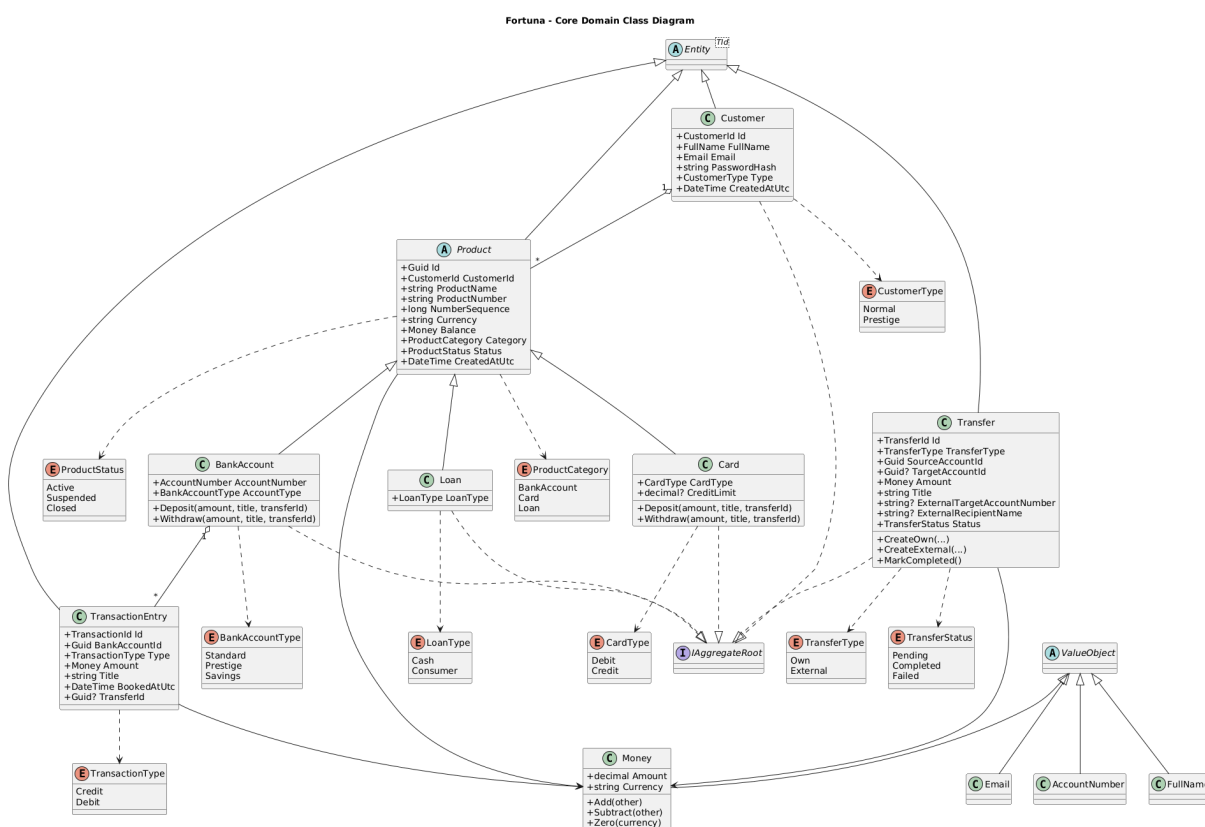
3.2 CQRS i rozdzielenie zapisu od odczytu

Operacje zmieniające stan systemu są realizowane przez komendy, np. `RegisterCustomerCommand`, `CreateTransferCommand`, `DepositMoneyCommand`. Operacje odczytowe są realizowane przez zapytania, np. `GetDashboardQuery`.

Dashboard nie czyta bezpośrednio tabel transakcyjnych. Zamiast tego korzysta z bazy odczytu `FortunaReadDb`, która zawiera gotowe widoki danych: kafle produktów oraz zdarzenia osi czasu. Dzięki temu odczyt dashboardu jest prosty i szybki, a model zapisu może pozostać bliższy regułom domenowym.

4 Model domenowy

Główne pojęcia domenowe to klient, produkty finansowe, pieniądze, transakcje i przelewy. Produkty finansowe mają wspólną klasę bazową `Product`, po której dziedziczą `BankAccount`, `Card` i `Loan`. Klient jest właścicielem produktów, a konto lub karta mogą rejestrować operacje wpłat i wypłat.



Rysunek 8: Diagram klas domenowych.

4.1 Agregaty i value objecty

Model używa podejścia DDD. Agregatami są między innymi `Customer`, `BankAccount`, `Card`, `Loan` i `Transfer`. Value objecty reprezentują dane z walidacją i znaczeniem domenowym, np. `Money`, `Email`, `FullName`, `AccountNumber`.

- `Money` przechowuje kwotę i walutę oraz udostępnia operacje dodawania i odejmowania.
- `Email` reprezentuje adres e-mail klienta i jest używany przy rejestracji oraz logowaniu.
- `FullName` rozdziela imię i nazwisko klienta.
- `AccountNumber` reprezentuje numer rachunku bankowego.

4.2 Zdarzenia domenowe

Encje domenowe generują zdarzenia, które są następnie zapisywane w tabeli `OutboxMessages`. Przykładowe zdarzenia to:

- `ProductCreatedDomainEvent`,
- `MoneyDepositedDomainEvent`,
- `MoneyWithdrawnDomainEvent`,
- `TransferCreatedDomainEvent`,
- `TransferCompletedDomainEvent`.

Zdarzenia są podstawą aktualizacji modelu odczytu.

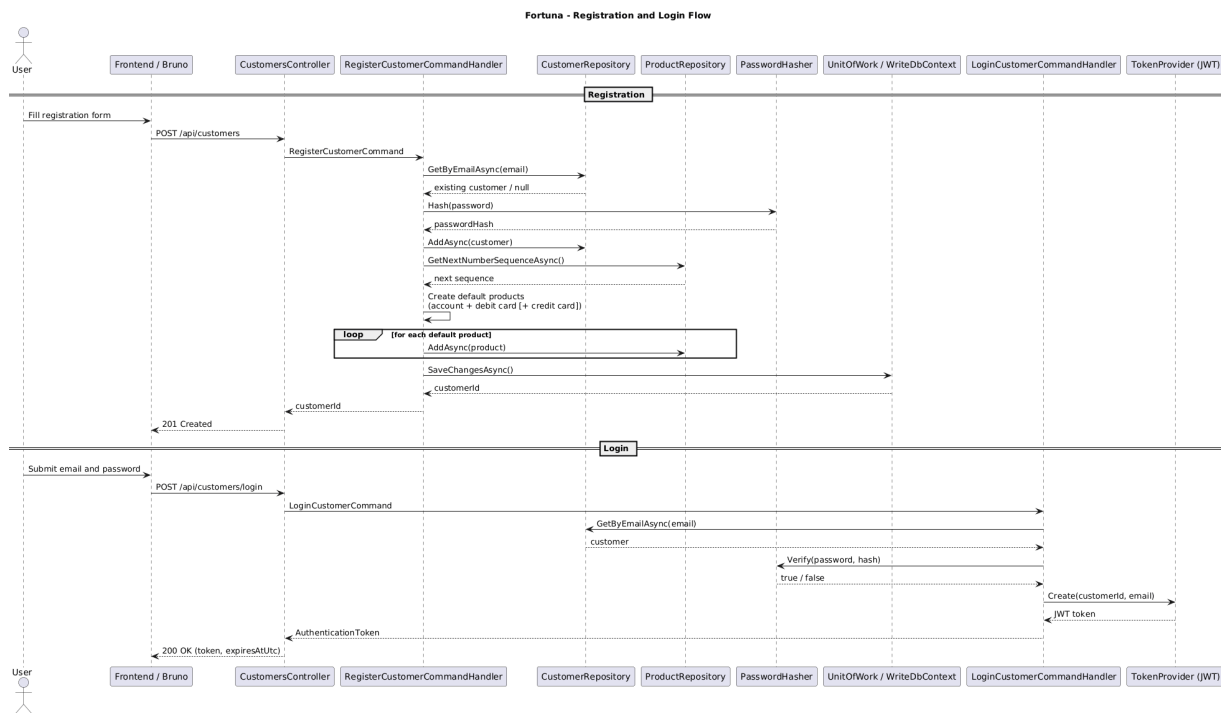
5 Działanie backendu

5.1 Rejestracja i logowanie

Rejestracja jest obsługiwana przez `CustomersController` i `RegisterCustomerCommandHandler`. Handler sprawdza długość hasła, waliduje unikalność e-maila, hashuje hasło algorytmem PBKDF2 i zapisuje nowego klienta. Następnie tworzy produkty startowe:

- konto standardowe albo prestiżowe,
- kartę debetową,
- dodatkową kartę kredytową dla klienta typu `Prestige`.

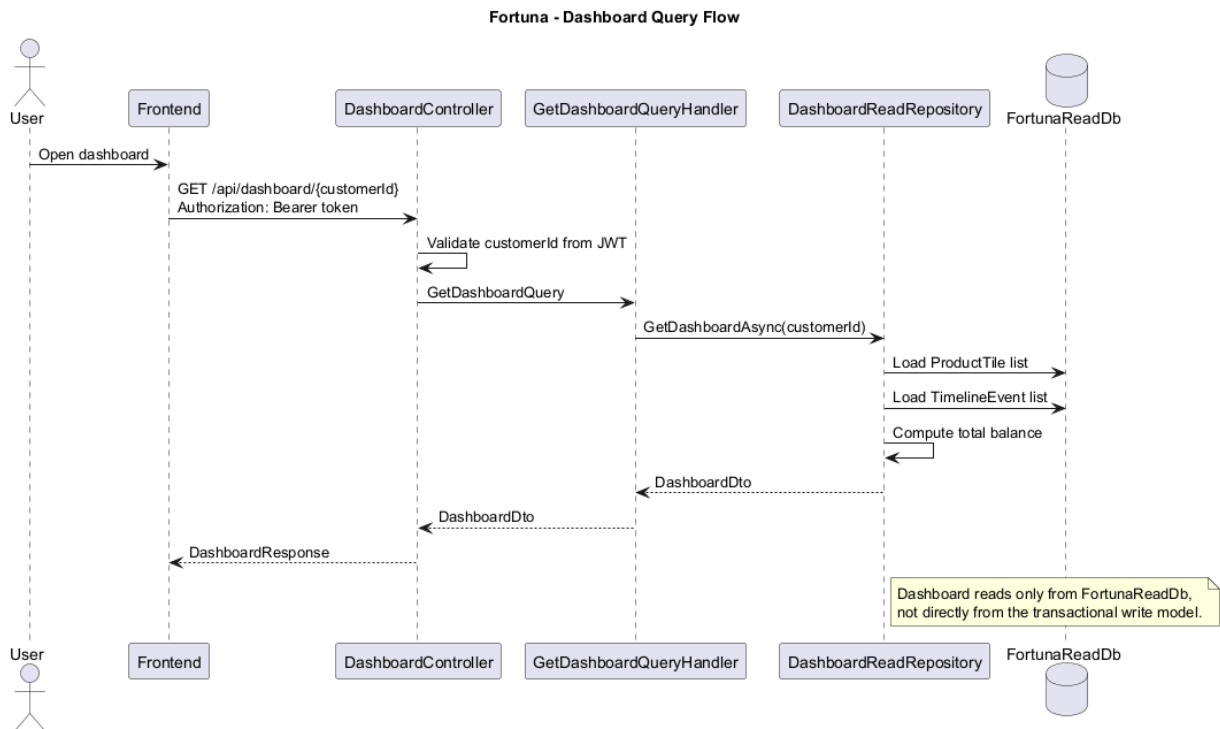
Logowanie jest obsługiwane przez `LoginCustomerCommandHandler`. Handler pobiera klienta po adresie e-mail, sprawdza hasło i generuje token JWT. Token zawiera identyfikator klienta, który później służy do autoryzacji zapytań.



Rysunek 9: Przepływ rejestracji i logowania.

5.2 Pobieranie dashboardu

Frontend wysyła żądanie `GET /api/dashboard/{customerId}` z tokenem JWT. Kontroler porównuje identyfikator z adresu URL z identyfikatorem zapisanym w tokenie. Jeśli identyfikatory są różne, zwracany jest `Forbidden`. W przeciwnym razie `GetDashboardQueryHandler` pobiera dane z `DashboardReadRepository`.



Rysunek 10: Przepływ pobierania dashboardu.

5.3 Wykonywanie przelewu

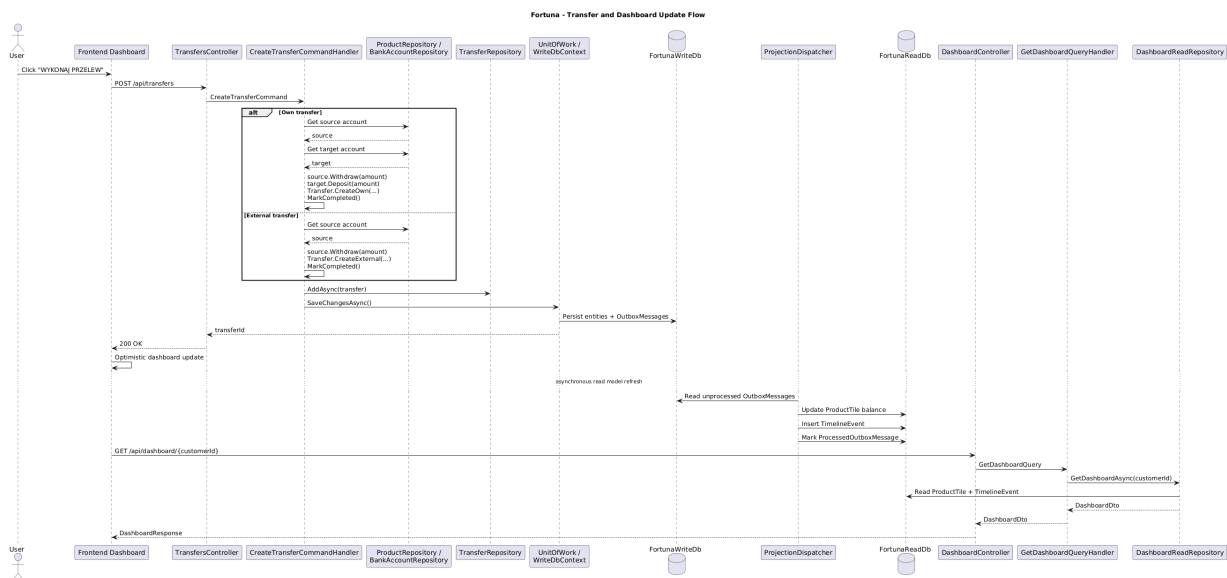
Przelew jest obsługiwany przez `TransfersController` i `CreateTransferCommandHandler`. Handler odczytuje produkt źródłowy i sprawdza, czy należy do zalogowanego klienta. Następnie rozróżnia przelew własny i zewnętrzny.

Dla przelewu własnego system:

- pobiera produkt źródłowy i docelowy,
- sprawdza, czy oba należą do klienta,
- wykonuje wypłatę ze źródła,
- wykonuje wpłatę na cel,
- zapisuje przelew i oznacza go jako zakończony.

Dla przelewu zewnętrznego system:

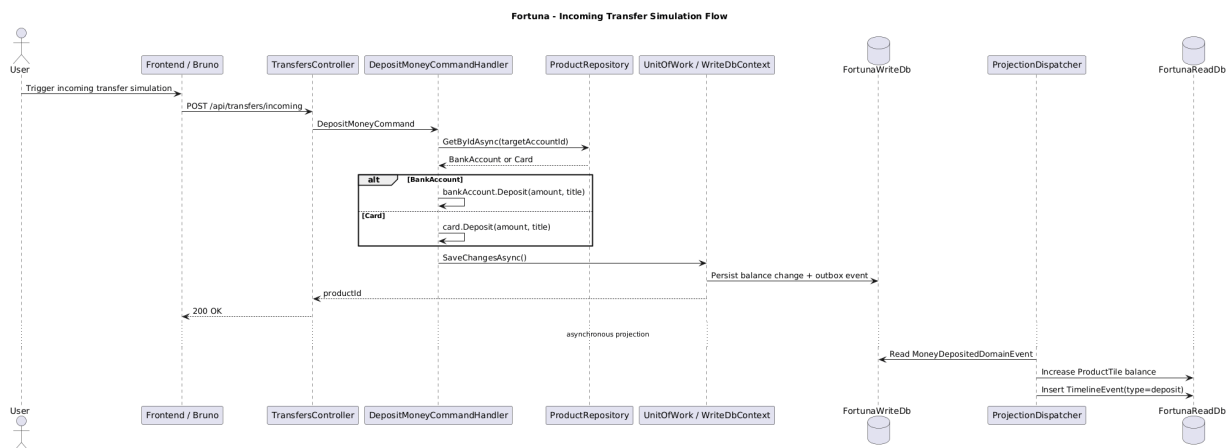
- wymaga numeru rachunku odbiorcy i nazwy odbiorcy,
- wykonuje wypłatę z produktu źródłowego,
- zapisuje przelew zewnętrzny,
- oznacza przelew jako zakończony.



Rysunek 11: Przelew i aktualizacja dashboardu.

5.4 Symulacja przelewu przychodzącego

Endpoint POST /api/transfers/incoming jest oznaczony jako AllowAnonymous. Służy do symulacji zewnętrznego wpływu środków, np. przy testowaniu przez Bruno. Komenda DepositMoneyCommand pobiera produkt docelowy, sprawdza, czy obsługuje wpłatę, zwiększa saldo i zapisuje zmiany.



Rysunek 12: Symulacja przelewu przychodzącego.

6 Mechanizm Outbox i projekcje

Podczas SaveChangesAsync w WriteDbContext system przegląda encje posiadające zdarzenia domenowe. Każde zdarzenie jest serializowane do JSON i zapisywane jako rekord w tabeli OutboxMessages. Po zapisaniu zmiany zdarzenia są czyszczone z encji.

Proces w tle OutboxMessageProcessor cyklicznie pobiera nieprzetworzone wiadomości z bazy

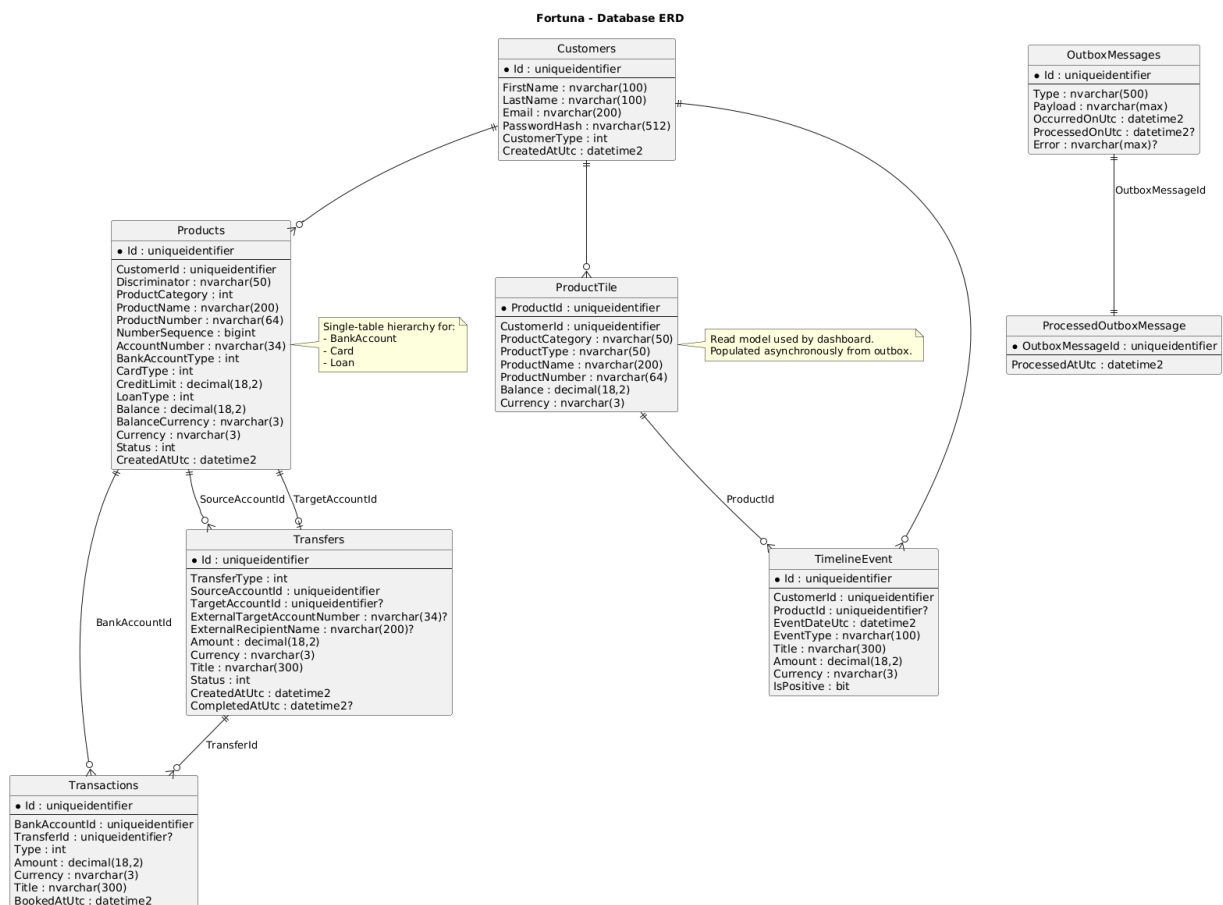
zapisu. Dla każdej wiadomości wywoływany jest `ProjectionDispatcher`, który aktualizuje bazę odczytu. Po poprawnym przetworzeniu rekord w `OutboxMessages` otrzymuje `ProcessedOnUtc`, a w bazie odczytu zapisywany jest wpis `ProcessedOutboxMessage`.

Projekcje obsługują między innymi:

- utworzenie kafła produktu po `ProductCreatedDomainEvent`,
- zwiększenie salda i dodanie zdarzenia osi czasu po `MoneyDepositedDomainEvent`,
- zmniejszenie salda i dodanie zdarzenia osi czasu po `MoneyWithdrawnDomainEvent`.

7 Schematy baz danych

System używa dwóch logicznych baz danych SQL Server: bazy zapisu `FortunaWriteDb` i bazy odczytu `FortunaReadDb`. Schemat jest pokazany na diagramie ERD.



Rysunek 13: Diagram ERD baz danych.

7.1 Baza zapisu

Baza zapisu przechowuje stan transakcyjny systemu i jest zarządzana przez `WriteDbContext`. To ona jest źródłem prawdy dla operacji domenowych.

Tabela	Opis
Customers	Dane klientów: identyfikator, imię, nazwisko, e-mail, hash hasła, typ klienta i data utworzenia. E-mail jest unikalny.
Products	Wspólna tabela dla produktów finansowych. EF Core używa dziedziczenia typu single-table hierarchy z kolumną Discriminator . W tabeli znajdują się konta, karty i kredyty.
Transactions	Zapisy księgowe dla wpłat i wypłat. Zawierają produkt, typ transakcji, kwotę, walutę, tytuł, datę księgowania i opcjonalny identyfikator przelewu.
Transfers	Przelewy własne i zewnętrzne. Zawierają źródło, opcjonalny cel wewnętrzny, dane odbiorcy zewnętrznego, kwotę, status oraz daty utworzenia i zakończenia.
OutboxMessages	Zserializowane zdarzenia domenowe oczekujące na projekcję do bazy odczytu.

7.2 Baza odczytu

Baza odczytu jest zoptymalizowana pod dashboard. Nie odtwarza pełnego modelu domenowego, tylko gotowe struktury potrzebne do wyświetlenia interfejsu.

Tabela	Opis
ProductTile	Kafle produktów widoczne na dashboardzie. Zawiera produkt, klienta, kategorię, typ, nazwę, numer, saldo i walutę.
TimelineEvent	Zdarzenia widoczne w panelu aktywności. Zawiera klienta, opcjonalny produkt, datę, typ zdarzenia, tytuł, kwotę, walutę i informację, czy zdarzenie zwiększa saldo.
ProcessedOutboxMessage	Identyfikatory wiadomości Outbox już przetworzonych przez projekcję. Chroni przed wielokrotnym przetworzeniem tego samego zdarzenia.

7.3 Najważniejsze relacje

- Jeden klient ma wiele produktów.
- Produkt typu konto bankowe może mieć wiele transakcji.
- Przelew ma produkt źródłowy i opcjonalny produkt docelowy.
- Transakcja może być powiązana z przelewem.
- Jeden klient ma wiele kafli produktu i zdarzeń osi czasu w modelu odczytu.
- Rekord Outbox może odpowiadać jednemu rekordowi **ProcessedOutboxMessage**.

8 Frontend

Frontend znajduje się w katalogu `frontend/src`. Aplikacja używa React Router do routingu, Redux Toolkit do stanu autoryzacji oraz TanStack Query do pobierania danych dashboardu.

8.1 Struktura

Katalog	Opis
<code>app</code>	Konfiguracja routera, klient API, providery i store Redux.
<code>features/auth</code>	Logowanie, rejestracja, sesja użytkownika i slice autoryzacji.
<code>features/dashboard</code>	Pobieranie danych dashboardu, typy i prezentacja produktów.
<code>features/transfers</code>	API przelewów, typy formularza i lokalna aktualizacja dashboardu po przelewie.
<code>components</code>	Komponenty widoków: landing page, logowanie, rejestracja, dashboard i przelewy.
<code>views</code>	Komponenty stron routingu: landing, login, create account, dashboard.
<code>shared/ui</code>	Wspólne komponenty UI, np. Button, Input, InfoPopup.

8.2 Komunikacja z API

Frontend używa funkcji `apiRequest`. Rejestracja i logowanie wysyłają żądania anonimowe. Dashboard i przelewy dodają nagłówek `Authorization`. Po wykonaniu przelewu frontend wykonuje lokalną, optymistyczną aktualizację danych, a następnie może pobrać odświeżony dashboard z read modelu.

9 Bezpieczeństwo i walidacja

Backend używa JWT Bearer Authentication. Endpointy kont, dashboardu i przelewów wymagają zalogowania. Kontroler dashboardu dodatkowo sprawdza, czy klient próbuje pobrać własne dane, porównując identyfikator z URL z identyfikatorem z tokenu.

Hasła nie są przechowywane jawnie. Implementacja `Pbkdf2PasswordHasher` zapisuje hash hasła. Rejestracja wymaga hasła o długości co najmniej 8 znaków, a logowanie zwraca błąd autoryzacji dla niepoprawnego e-maila albo hasła.

Walidacja reguł biznesowych znajduje się w handlerach i modelu domenowym. Przykładowo przelew zewnętrzny wymaga numeru rachunku i nazwy odbiorcy, przelew własny wymaga innego produktu docelowego niż źródłowy, a operacje wpłat i wypłat są obsługiwane tylko dla kont oraz kart.

10 Podsumowanie

Fortuna jest aplikacją wielowarstwową, w której frontend React komunikuje się z backendem ASP.NET Core przez REST API. Backend realizuje Clean Architecture i DDD, a przypadki użycia są rozdzielone zgodnie z CQRS. Model zapisu przechowuje pełny stan domenowy i generuje zdarzenia, natomiast osobny model odczytu udostępnia dane zoptymalizowane dla dashboardu. Mechanizm Outbox zapewnia asynchroniczne i kontrolowane przenoszenie zmian z modelu zapisu do modelu odczytu.